

A Serially-Streamed KV-Cache Compression and Eviction Coprocessor on GF180MCU, Taken from RTL to GDS by an LLM Agent

Chaithu Talasila
Longhorn Silicon (Team A71)
The University of Texas at Austin
Austin, TX, USA
talasila.chaithu1@gmail.com

Abstract—The key/value (KV) cache is the dominant memory- and bandwidth-consumer of autoregressive large language model (LLM) decoding: it grows linearly with context length and is re-read in its entirety at every generated token. We present a compact coprocessor that moves three KV-cache maintenance operations off the host and into dedicated silicon: (i) rotated INT3 value compression using a Walsh–Hadamard transform (WHT) and per-token scaling, (ii) H2O heavy-hitter token-importance scoring with single-victim eviction, and (iii) a divide-free per-tile precision gate. The block is driven entirely over a four-wire SPI slave, keeping its pad footprint small enough for a shared-reticle slot. We harden the design as a core-only pre-integration macro on the GlobalFoundries 180 nm (GF180MCU) open PDK using the LibreLane flow. The implementation occupies $502.89 \times 520.81 \mu\text{m}$ (0.262 mm^2 , 7,080 standard cells) and passes signoff with zero violations across Magic DRC, KLayout DRC, routing DRC, LVIS, antenna, and layout XOR. Distinctively, the entire RTL-to-GDS path—including diagnosing and repairing a pin-naming audit failure, reordering pads for the assigned padframe slot, and reconciling the delivered floorplan—was carried out by a large language model (LLM) coding agent, with every step reproduced in continuous integration. We report the architecture, the agentic design methodology, the physical results, and an explicit account of what is and is not yet verified.

Index Terms—KV cache, LLM inference, hardware accelerator, quantization, Walsh–Hadamard transform, GF180MCU, open-source EDA, LLM agents for chip design.

I. Introduction

Autoregressive decoding in transformer-based LLMs caches the per-layer keys and values of all previously seen tokens so that self-attention need not be recomputed. This KV cache scales as $O(L \cdot d)$ in the number of cached tokens L and the per-token channel width d , and it is streamed in full for every newly generated token. In deployment the KV cache, rather than the arithmetic, becomes the binding constraint on memory footprint and off-chip bandwidth, and the host expends a disproportionate share of its energy moving the cache rather than computing on it [1], [3].

A large body of algorithmic work reduces this cost along two axes: eviction, which discards tokens predicted to be unimportant (e.g. H2O’s heavy-hitter oracle [1]), and quantization, which stores each cached value at low precision, often after a rotation that spreads outliers across

channels so that uniform quantizers behave well [2]–[4]. These techniques are almost always evaluated in software. We instead ask what a small, dedicated silicon block that performs this cache maintenance at the edge of memory looks like, and—because the target is a Track D (AI/LLM-for-circuits) shared-reticle tapeout—whether such a block can be produced end-to-end by an LLM coding agent.

Contributions.

- A serially-streamed KV-cache coprocessor that unifies rotated INT3 value compression, H2O token-importance/eviction, and a divide-free precision gate behind a single four-wire SPI slave (Section III).
- An agentic RTL-to-GDS methodology in which an LLM coding agent hardens the design, diagnoses and repairs a physical pin-naming audit, reorders pads for the assigned padframe slot, and reconciles the delivered floorplan—each step reproduced in CI (Section IV).
- A signoff-clean GF180MCU implementation (0.262 mm^2 , all checks zero) with a calibrated statement of verification scope (Sections V–VIII).

II. Related Work

KV-cache eviction. H2O [1] observes that a small set of “heavy-hitter” tokens accounts for most attention mass and can be identified by accumulating per-token attention weights; low-mass tokens are evicted with little quality loss. Our token-importance unit (TIU) implements this accumulate-then-argmin policy directly in hardware.

Rotated low-bit KV quantization. QuaRot [2] and related methods apply an orthogonal (Hadamard) rotation before quantization so that per-channel outliers are redistributed, enabling aggressive uniform quantization; KVQuant [3] and KIVI [4] specialize such ideas to the KV cache. Our value path (KVE) uses a Walsh–Hadamard transform followed by per-token amax scaling and INT3 quantization, a synthesizable, division-light realization of the same principle.

LLM agents for chip design. Language models have been applied to RTL generation and EDA assistance [5]. We push further along the flow: rather than generating a

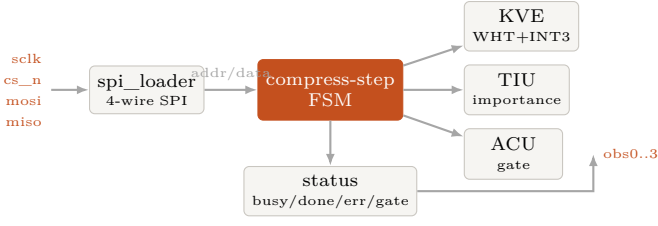


Fig. 1. Coprocessor block diagram. A four-wire SPI loader feeds a byte-addressed fabric; the compress-step FSM (highlighted) orchestrates the KVE, TIU, and ACU engines and exposes a four-bit status word on the observation pins.

snippet, an agent drives an open-source implementation flow [6] to a signoff-clean GDS and verifies its own output against the physical database.

III. Architecture

Fig. 1 shows the block. A host streams work in over a standard four-wire SPI slave (SCLK, CS_n, MOSI, MISO); an on-chip loader exposes a byte-addressed register fabric, and a compress-step finite-state machine (FSM) sequences three hardened engines. Only ten functional signals plus power leave the block, which is what allows it to fit a 1/8 shared-reticle slot.

A. KVE: rotated INT3 value compression

For each cached value token $\mathbf{v} \in \mathbb{R}^d$ (with d a power of two), the KVE applies a forward Walsh–Hadamard transform $\mathbf{r} = \mathbf{H}_d \mathbf{v}$, computes a per-token maximum absolute value $s = \max_i |r_i|$, and quantizes to a signed 3-bit code

$$c_i = \text{clip}\left(\left\lfloor \frac{r_i}{s} (2^{b-1} - 1) \right\rfloor, -4, 3\right), \quad b = 3, \quad (1)$$

emitting the d INT3 codes together with a single fp16 scale s ; a rotated fp16 reconstruction $\hat{\mathbf{v}}$ is produced for read-back. The transform spreads outliers across channels so the uniform INT3 grid is used efficiently, echoing rotated-quantization results in software [2], [3]. The datapath is fully synthesizable and bit-exact against a behavioral reference; the combinational transform path is the timing-critical path and sets the operating period.

B. TIU: H2O token importance and eviction

The TIU maintains a per-slot importance score by accumulating host-supplied attention masses. On request it (i) emits a keep-tier bitmap by thresholding each slot’s score and (ii) selects a single eviction victim as the minimum-mass slot via a serialized argmin. This is a direct hardware realization of the H2O heavy-hitter policy [1].

C. ACU: divide-free precision gate

The ACU decides, per tile, whether the reconstructed values are “peaky” enough to warrant retaining full fp16 precision. It compares a scaled maximum against the tile sum using only shifts, adds, and comparisons—no division—and raises a single gate_fp16 bit.

TABLE I
Byte-addressed register map (fp16 little-endian).

Address	Region	Dir.
0x0300	W — per-token attention mass	write
0x0400	V — value tokens (fp16)	write
0x0800	CODES — rotated INT3 codes	read
0x0A00	SCALE — per-token fp16 scale	read
0x0C00	$\hat{\mathbf{V}}$ — rotated reconstruction	read
0x0002	DECISION — {gate, evict slot, keep}	read
0x0001	STATUS mirror	read

D. Serial interface and register map

The loader implements SPI mode 0 (CPHA=0), MSB-first: a command byte (WRITE/READ/START/STATUS) is followed by a 16-bit address and payload. The host writes value tokens and attention masses, pulses START, polls STATUS, and reads back the compressed records and decision byte (Table I). Because no output-only digital pad exists in the GF180 IO library, MISO and the four observation bits are declared bidirectional and used as outputs.

One compress step therefore proceeds as: stream L value tokens and masses in; the KVE compresses every token to codes, scale, and reconstruction; the TIU scores importance and selects a victim; the ACU sets the precision bit; the host polls STATUS and reads the records back—all over the same four wires.

IV. Design Methodology: Agentic RTL-to-GDS

The distinguishing feature of this work (and its Track D thesis) is that the implementation flow was executed by an LLM coding agent rather than by hand. The agent operated on the repository directly: reading the design and flow configuration, editing HDL and YAML, running the physical-design flow inside a container against the GF180MCU PDK, and—crucially—verifying its own output against the physical database rather than asserting success.

Two episodes are illustrative. First, a shared-reticle pin audit compares the integrator’s expected pin names character-for-character against the text labels in the submitted GDS. An initial audit failed on six names because the status port had been elaborated as a bus (obs_out[3:0], labelled obs_out[0]...) while the pin list declared scalars (obs0...obs3), and because power had hardened as VDD/VSS while the list declared vdd/vss. The agent localized the root cause to naming (not logic), split the bus into four scalar ports preserving bit order, aligned the power nets, re-hardened, and then re-scanned the resulting GDS to confirm the labels obs0...obs3 and VDD/VSS were present and no obs_out[*] remained. Second, on receiving the padframe floorplan for a 1/8 “BV” slot—which requires the chip-wide common ground on pin 1—the agent reordered the pin list so that ground

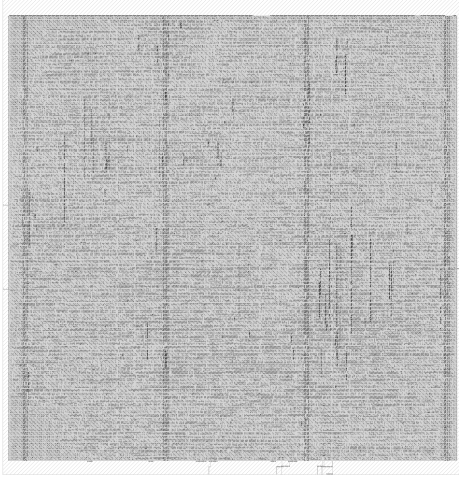


Fig. 2. Routed GF180MCU layout of the core-only coprocessor (lambda_kv_coproc), $502.89 \times 520.81 \mu\text{m}$.

TABLE II
GF180MCU implementation and signoff.

Parameter	Value
PDK / cells	gf180mcuD, 180 nm / mcu7t5v0 (5 V)
Die area	$502.89 \times 520.81 \mu\text{m}$ (0.262 mm ²)
Standard cells	7,080 (~62% utilization)
Clock period	300 ns (setup/hold clean)
Magic DRC / KLayout DRC	0 / 0
Routing DRC / antenna	0 / 0
LVS / layout XOR	0 / 0

leads, and reconciled the delivered floorplan against the pin list (both offered orientations matched exactly).

Every such step is reproduced by continuous integration, which reruns the self-checking functional test and the full hardening on each push. The agent’s work is thus auditable rather than asserted: a third party can re-execute the flow and obtain the same signoff.

V. Implementation Results

The design is hardened as a core-only pre-integration macro (no padding or seal ring) with the LibreLane Classic flow [6] on the open GF180MCU PDK [7], using the 5 V gf180mcu_fd_sc_mcu7t5v0 standard cells. Fig. 2 shows the routed layout; Table II summarizes the physical metrics and signoff. All six signoff checks report zero violations, and static timing closes with no setup or hold violations at the target period.

The 300 ns period (~3.3 MHz) is set by the combinational Walsh–Hadamard value path on this low-cost 180 nm node; it is a deliberate area-for-speed trade for a streaming maintenance block that is not on the model’s arithmetic critical path.

VI. Verification

Functional correctness is established by a self-checking testbench that exercises the complete SPI contract at

TABLE III
BV-slot pin assignment (12 of 16 pins used).

Pin	Signal	Pad / type
1	VSS	W12 / ground
2	clk	W13 / Schmitt in
3–6	rst_n, spi_sclk/cs_n/mosi	W14–17 / in
7	spi_miso	W18 / bidir
8–11	obs0–obs3	W19–22 / bidir
12	VDD	N01 / power

the workshop tile size ($d=2$, $L=2$): it writes importance masses and value tokens, issues START, polls STATUS to completion, and reads back the decision byte, keep bitmap, and KVE codes/scale. The checks are deterministic—FSM completion with the error flag clear, eviction victim equal to the argmin of the masses, the keep bitmap equal to the per-slot threshold test, INT3 codes within $[-4, 3]$, and stable scale read-back—and all pass. Layout-versus-schematic (LVS) ties the extracted layout back to the synthesized netlist, so the GDS is a faithful realization of the verified RTL. Both the functional test and the hardening are re-executed in CI on every push.

VII. Shared-Reticle Integration

For the shared-reticle flow the team submits a core-only block plus a pin list; the organizers generate the padframe, sized to the core area, and return a floorplan (DEF) that the core is hardened into. Our twelve pins map onto a 1/8, 16-pin “BV” slot (Table III); the four unused slots are auto-populated with analog pads by the integrator. The final GDS is pulled from the project’s default branch, combined with the other projects, and forwarded to the channel partner—so integration is fully scripted and requires no manual editing of the submitted layout.

VIII. Limitations and Future Work

We state verification scope precisely. Correctness is established at the RTL plus physical-signoff level: RTL simulation of the SPI contract together with LVS. We have not run gate-level simulation of the post-layout netlist, and the SPI testbench is a targeted deterministic self-check rather than exhaustive coverage; bit-exactness of the KVE against its behavioral reference is covered by block-level testbenches maintained separately. Pad-level behavior (the core driving the physical pads through their output-enable and configuration terminals) is handled by the integrator’s scripted padframe wiring and is not self-verified here. Finally, the design is pre-fabrication: no silicon measurements exist. Natural next steps are gate-level and post-layout timing simulation, a wider randomized functional suite, and—after fabrication—bring-up over the same SPI link.

IX. Conclusion

We presented a compact, serially-streamed KV-cache coprocessor that performs rotated INT3 value compression, H2O token-importance/eviction, and a divide-free precision gate behind a four-wire SPI slave, and hardened it to a signoff-clean 0.262mm² macro on the open GF180MCU node. The design was carried from RTL to GDS by an LLM coding agent that repaired a physical pin-naming audit, adapted to the assigned padframe slot, and verified its own output against the GDS—with every step reproduced in continuous integration. The result argues that agent-driven, open-source silicon implementation can produce auditable, tapeout-ready blocks for shared-reticle programs.

Reproducibility

The RTL, flow configuration, hardened GDS, and this paper are open at <https://github.com/LonghornSilicon/gf180-longhorn-tiu>. Functional test: `make sim-coproc`; hardening: `make coproc`.

References

- [1] Z. Zhang et al., “H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.
- [2] S. Ashkboos et al., “QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.
- [3] C. Hooper et al., “KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.
- [4] Z. Liu et al., “KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.
- [5] M. Liu et al., “ChipNeMo: Domain-Adapted LLMs for Chip Design,” *arXiv:2311.00176*, 2023.
- [6] M. Shalan and T. Edwards, “Building OpenLANE: A 130nm OpenROAD-based Tapeout-Proven Flow,” in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2020.
- [7] GlobalFoundries and Google, “GF180MCU Open Source PDK,” <https://github.com/google/gf180mcu-pdk>, 2022.